

Visual Gun Detection System Technical Report

Fares Hazem (ID: 20221443356)
Ali Ashraf (ID: 2103106)
Ahmed Dawood (ID: 20221454408)
Ahmed Yousri (ID: 2103108)
Ahmed Ashraf (ID: 2103134)

May 3, 2025

1 Abstract

This report details the implementation and comparison of techniques within a Visual Gun Detection System, developed as a hands-on computer vision project. The system detects guns in static images and simulated real-time video feeds using color-based segmentation (K-means clustering), morphological image processing, interest point detection (Harris Corners/FAST, SIFT, ORB), feature description (FREAK, SIFT, ORB), and feature matching (Brute-Force Matcher with ratio test). The project provides practical experience in applying and comparing these techniques for security applications like surveillance and scanning.

2 Introduction

The objective of this project was to design, implement, and compare methods for a visual gun detection system. The core techniques employed and compared include:

- **Color-based segmentation:** Using K-means clustering to isolate potential gun regions based on color.
- **Interest Point Detection and Description:** Utilizing Harris Corner Detection coupled with FREAK descriptors as the primary method. Additionally, SIFT and ORB detectors/descriptors were implemented for comparison.
- **Feature Matching:** Comparing extracted FREAK features against a database of known gun descriptors using a Brute-Force Matcher (Hamming distance) and Lowe's ratio testing for the primary pipeline.
- **Morphological Operations:** Refining segmented images using closing and opening operations.

The system was developed to process static images, allowing for visualization and comparison of different feature detectors (Harris+FREAK, SIFT, ORB). It was also extended to perform detection in a simulated real-time video feed using the Harris+FREAK pipeline.

3 Methodology

The gun detection process follows these key steps:

3.1 Data Collection

A dataset of static images containing both guns and non-guns was gathered, as specified in the project requirements (target 30-50 images). For real-time simulation and testing the matching pipeline, a video file (Video.mp4) was used. Reference gun features were pre-computed and stored.

3.2 Preprocessing

Input images (and video frames) were consistently resized to 400x300 pixels. Median filtering was applied to reduce noise. Based on implementation choices, a kernel size of 3x3 was used in the static image processing notebook (`Visual_Gun_Detection_System.ipynb`) and 5x5 in the video processing script (`Gun_Detection_With_Camera_Feed.py`).

3.3 Color Segmentation

K-means clustering with K=3 clusters was implemented on the RGB image to segment it based on color properties. Pixels were reassigned to their corresponding cluster center values. A binary mask was generated by thresholding the segmented image, specifically isolating the darkest cluster, under the assumption that it often corresponds to gun regions.

3.4 Morphological Processing

To enhance the binary masks derived from segmentation, morphological operations were applied. Morphological closing (to fill small holes) followed by opening (to remove small noise specks) were performed, both using a 5x5 rectangular kernel. This step aimed to produce cleaner, more contiguous blobs representing potential objects.

3.5 Feature Extraction

Multiple feature detection and description methods were implemented:

- **Harris+FREAK (Primary Pipeline):** For the main detection pipeline (especially in the video feed), Harris corner detection was potentially used initially, followed by FAST (Features from Accelerated Segment Test) detection on the grayscale image, guided by the morphological mask. FREAK (Fast Retina Keypoint) descriptors were then computed for these keypoints.
- **SIFT:** The Scale-Invariant Feature Transform (SIFT) detector and descriptor were implemented in the notebook for comparison. SIFT is known for its robustness to scale and rotation changes.
- **ORB:** Oriented FAST and Rotated BRIEF (ORB) detector and descriptor were also implemented in the notebook. ORB is designed as an efficient and rotation-invariant alternative to SIFT and SURF.

Keypoints and descriptors for all methods were computed and saved (or used directly) for visualization and matching.

3.6 Feature Matching (Harris+FREAK Pipeline)

For gun detection in the video feed script (`Gun_Detection_With_Camera_Feed.py`), the FREAK descriptors extracted from the current frame were matched against pre-loaded FREAK descriptors from reference gun images stored in `Processing/GunFeatures`. A Brute-Force Matcher (`cv2.BFMatcher`) using Hamming distance (appropriate for binary descriptors like FREAK and ORB) was employed. To filter reliable matches, Lowe's ratio test was applied to the k=2 nearest neighbors with a ratio threshold of 0.7.

3.7 Classification (Harris+FREAK Pipeline)

A frame from the video feed was classified as containing a gun if the total number of good matches (passing the ratio test) against the reference FREAK descriptors reached or exceeded an absolute threshold of 20 matches.

4 Implementation Details

The system was implemented in Python using several key libraries:

- **OpenCV (cv2):** Core library for image/video I/O, resizing, median blur, color space conversions, K-means (used alongside scikit-learn’s implementation or OpenCV’s own), morphological operations, Harris corners, FAST, SIFT, ORB, FREAK, and `BFMatcher`.
- **NumPy:** Essential for numerical processing and array manipulation, especially with image data.
- **Scikit-learn (`sklearn.cluster.KMeans`):** Used for K-means color segmentation in the notebook.
- **Matplotlib:** Employed in the notebook for visualizing images, masks, and detected keypoints from different methods.
- **OS, Glob, JSON:** Used for file system interactions (creating directories, finding files) and potentially handling data serialization.

The project work is primarily contained within two Python files:

1. **Visual_Gun_Detection_System.ipynb:** A Jupyter notebook detailing the steps for static image processing, including preprocessing, segmentation, morphological operations, feature extraction (Harris+FREAK, SIFT, ORB), and crucially, visualization of intermediate steps and comparison of keypoints detected by the different methods.
2. **Gun_Detection_With_Camera_Feed.py:** A Python script implementing the pipeline for simulated real-time detection using a video file. It incorporates preprocessing, segmentation, feature extraction (FAST+FREAK), feature matching against stored references, classification, and overlays the detection results onto the video frames.

5 Visualization and Qualitative Results

The Jupyter notebook (`Visual_Gun_Detection_System.ipynb`) provides significant visual feedback on the different stages of the processing pipeline for static images. This includes:

- Side-by-side comparisons of original images with their preprocessed (resized and filtered) versions.
- Visualization of the K-means color segmentation results and the generated binary masks after morphological processing.
- Overlays of detected keypoints onto the images for the Harris+FREAK, SIFT, and ORB methods, allowing for a qualitative visual comparison of where each detector finds interest points.

The real-time script (`Gun_Detection_With_Camera_Feed.py`) provides dynamic qualitative results by processing a video file frame-by-frame. It displays the video with detected keypoints (from FAST+FREAK) drawn and textual labels (“Gun Detected” in red or “No Gun” in green) overlaid, indicating the system’s classification decision in real-time based on the feature matching threshold.

6 Challenges Faced

Several challenges were encountered during the development and are inherent in this approach:

- **Color Segmentation Sensitivity:** K-means clustering performance is dependent on the chosen K value and initial centroids. Relying on the darkest cluster for gun segmentation is a heuristic that fails for non-dark guns or under challenging lighting conditions (e.g., shadows, glare).
- **Feature Matching Thresholding:** Using a fixed absolute threshold (20 matches) for classification might not generalize well across different gun types, distances, or levels of occlusion. The optimal threshold can be scene-dependent.
- **Descriptor Robustness Trade-offs:** While FREAK and ORB are computationally efficient, they might be less robust to extreme viewpoint or illumination changes compared to SIFT. Selecting the right feature depends on the application’s constraints (speed vs. accuracy).
- **Morphological Parameter Tuning:** Finding the optimal kernel size and sequence of morphological operations (closing, opening) requires experimentation to effectively remove noise and connect regions without excessively distorting the object’s shape.
- **Reference Descriptor Quality:** The performance of the matching stage heavily relies on the quality and representativeness of the pre-computed reference gun descriptors.

7 Conclusion and Future Work

This project successfully implemented and explored a visual gun detection system using traditional computer vision techniques. Keypoint detection methods (Harris/FAST, SIFT, ORB) combined with descriptors (FREAK, SIFT, ORB) were implemented, with a full pipeline utilizing Harris/FAST+FREAK demonstrated for simulated real-time detection via feature matching. The comparison visualizations provided insights into the behavior of different feature detectors.

Future enhancements could focus on:

- **Robustness:** Improving handling of various lighting conditions, occlusions, and backgrounds, possibly through adaptive thresholding or more sophisticated segmentation techniques. Exploring multi-color gun detection and handling scenarios with multiple guns.
- **Adaptive Thresholding:** Developing methods to dynamically adjust the feature matching threshold based on image statistics or context, rather than using a fixed value.